SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 3 OF 10

FastAPI Deep Dive

12 Questions | DI Internals | Pydantic v2 | Auth | Scaling | Testing

FastAPI Deep Dive

Q1: Why FastAPI over Flask or Django? What does it actually give you?

FastAPI is built specifically for async-first, high-performance APIs. The comparison is not just about speed — it's about what the framework forces you to do well by default.

Automatic validation: FastAPI uses Pydantic to validate request bodies, query params, path params, and headers. Flask/Django require you to manually validate or add plugins. With FastAPI, if validation fails, a structured 422 response is returned automatically.

Automatic OpenAPI docs: Every endpoint's schema, request body, response model, and error codes are auto-generated into a Swagger UI (/docs) and ReDoc (/redoc). In Flask/Django this requires manual maintenance or extra libraries.

Type safety: FastAPI uses Python type hints everywhere — not just for documentation, but to actually drive validation and serialization. The type hint IS the contract.

Native async support: Flask is WSGI — async requires hacks (flask async views exist but are bolted on). Django's async support is improving but the ORM is still largely synchronous. FastAPI is ASGI-native, async from the ground up.

Dependency injection: FastAPI's Depends() system is a first-class DI framework built into the framework — not a plugin. Auth, DB sessions, config, and feature flags all flow through this cleanly.

```
# Flask — manual validation, no type safety
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route('/user', methods=['POST'])
def create_user():
    data = request.get_json()
    if not data.get('email'):   # manual validation
        return jsonify({'error': 'email required'}), 400
    ...

# FastAPI — validation, docs, typing, DI — all automatic
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr
app = FastAPI()
class UserCreate(BaseModel):
    email: EmailStr
    name: str
@app.post('/user', response_model=UserOut, status_code=201)
async def create_user(payload: UserCreate, db: Session = Depends(get_db)):
    return await db.create_user(payload)
```

When Django wins: Admin interface, full ORM, batteries-included auth, and multi-page apps. Django is still the right choice for content-heavy apps. FastAPI is for pure API backends.

Q2: How does FastAPI's Dependency Injection system work internally?

FastAPI's DI is built on Python's inspect module. When a route is registered, FastAPI inspects the function signature, finds parameters annotated with Depends(), and builds a dependency graph. At request time, it resolves the graph — executing dependencies in topological order.

```
from fastapi import Depends, FastAPI, HTTPException
from typing import Annotated

app = FastAPI()
```

```
# A dependency is just a callable
async def get_db():
    db = SessionLocal()
    try:
        yield db          # generator-based — cleanup runs after response
    finally:
        db.close()

async def get_current_user(token: str, db = Depends(get_db)):
    user = await db.get_user_by_token(token)
    if not user:
        raise HTTPException(status_code=401, detail='Invalid token')
    return user

# Route depends on get_current_user, which depends on get_db
# FastAPI builds: get_db -> get_current_user -> route_handler
@app.get('/profile')
async def get_profile(user = Depends(get_current_user)):
    return user
```

yield dependencies: A dependency that yields (instead of returning) is a context manager. Code before yield runs before the route handler; code after yield runs after the response is sent. Perfect for DB sessions, transaction management, and resource cleanup.

Dependency caching: Within a single request, if the same dependency is declared multiple times (e.g., get_current_user used by two sub-dependencies), FastAPI calls it only once and caches the result. Override with use_cache=False when you need fresh resolution.

Class-based dependencies: Any callable works — including classes with __call__. This lets you parameterize dependencies: CommonQueryParams(page=1, size=20) creates a configurable dependency via the class constructor.

```
# Parameterized dependency via class
class Paginator:
    def __init__(self, page: int = 1, size: int = 20):
        self.page = page
        self.size = size
        self.offset = (page - 1) * size

@app.get('/items')
async def list_items(pagination: Annotated[Paginator, Depends()]):
    return await fetch_items(pagination.offset, pagination.size)
```

Q3: Explain Pydantic v2 — how does validation actually work?

Pydantic v2 rewrote the core in Rust (via pydantic-core). It is 5-50x faster than v1. Validation is now compiled — Python type hints are converted to a validation schema at class definition time, not at call time.

```
from pydantic import BaseModel, Field, field_validator, model_validator
from pydantic import EmailStr, HttpUrl
from typing import Annotated

class UserCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    email: EmailStr
    age: Annotated[int, Field(ge=0, le=150)]
    website: HttpUrl | None = None

    # Field-level validator
    @field_validator('name')
    @classmethod
    def name_must_not_be_blank(cls, v: str) -> str:
```

```
        if v.strip() == '':
            raise ValueError('name cannot be blank')
        return v.strip()   # validators can transform values

    # Model-level validator — sees all fields
    @model_validator(mode='after')
    def check_consistency(self) -> 'UserCreate':
        if self.age < 18 and self.website:
            raise ValueError('minors cannot have websites')
        return self
```

Validation modes: mode='before' runs before type coercion. mode='after' runs after all fields are validated and coerced. Use 'before' to normalize inputs (strip strings, lowercase emails); use 'after' for cross-field logic.

model_config: Replaces v1's class Config. Controls: from_attributes=True (ORM mode — read from objects, not dicts), strict=True (no coercion), populate_by_name=True (accept both field name and alias), json_schema_extra (add examples to OpenAPI).

```
from pydantic import ConfigDict

class UserOut(BaseModel):
    model_config = ConfigDict(
        from_attributes=True,    # read SQLAlchemy model attributes
        str_strip_whitespace=True,
    )
    id: int
    name: str
    email: str

# Now you can do: UserOut.model_validate(sqlalchemy_user_object)
```

Serialization: model.model_dump() returns a dict. model.model_dump_json() returns JSON bytes. Both support include/exclude/by_alias/exclude_none. Custom serializers via @field_serializer.

Q4: What is the FastAPI request lifecycle from TCP to response?

Understanding the full lifecycle helps you know exactly where to hook into for middleware, auth, validation errors, and response transformation.

```
1. TCP connection accepted by Uvicorn (ASGI server)
2. Uvicorn parses HTTP, calls app(scope, receive, send)
3. Starlette router matches the path to a route
4. Middleware stack executes (request phase): logging, auth, rate limit
5. FastAPI resolves dependency graph for the matched route
   - Dependencies execute in topological order
   - yield-based dependencies enter __aenter__
6. Request body is read and parsed by Pydantic
   - Type coercion applied
   - Validators run
   - 422 returned immediately if validation fails
7. Route handler (your async def function) executes
8. Return value passed to response_model for filtering/serialization
9. JSONResponse (or custom response class) serialized
10. Middleware stack executes (response phase)
11. yield-based dependencies: code after yield runs (cleanup)
12. Background tasks execute AFTER response is sent
13. Connection closed or kept alive (keep-alive)
```

Key insight: Background tasks run after the response is already sent to the client. The client does not wait for them. This makes them suitable for fire-and-forget operations like sending emails, logging analytics, or triggering webhooks.

Q5: How does middleware work in FastAPI / Starlette?

FastAPI middleware wraps the entire application. Each middleware is an ASGI app that calls the next middleware (or the actual app) and can modify the request/response or short-circuit entirely.

```
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
import time, uuid

class RequestLoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        request_id = str(uuid.uuid4())
        request.state.request_id = request_id
        start = time.time()

        try:
            response = await call_next(request)  # call the next layer
        except Exception as e:
            # Exceptions from the route bubble up here
            raise
        finally:
            duration = time.time() - start
            print(f'{request.method} {request.url.path} -> {response.status_code}
({duration:.3f}s)')

        response.headers['X-Request-ID'] = request_id
        return response

app.add_middleware(RequestLoggingMiddleware)

# Order matters: last added = outermost (executes first)
app.add_middleware(CORSMiddleware, allow_origins=['*'])
app.add_middleware(GZipMiddleware, minimum_size=1000)
app.add_middleware(RequestLoggingMiddleware)
# Execution order: Logging -> CORS -> GZip -> your routes
```

Low-level ASGI middleware: For performance-critical middleware (rate limiting, auth on every request), prefer raw ASGI middleware over BaseHTTPMiddleware. BaseHTTPMiddleware buffers the entire response body into memory — a problem for streaming responses.

```
# Raw ASGI middleware — no buffering
class RawMiddleware:
    def __init__(self, app):
        self.app = app
    async def __call__(self, scope, receive, send):
        if scope['type'] == 'http':
            # Modify scope (add auth user to state, etc.)
            pass
        await self.app(scope, receive, send)
```

Q6: How do you implement authentication and authorization in FastAPI?

FastAPI has no built-in auth — it gives you the primitives (security schemes, Depends) to build it cleanly. The idiomatic approach uses OAuth2PasswordBearer or HTTPBearer as a dependency that extracts the token, plus a get_current_user dependency that validates it.

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import JWTError, jwt
from datetime import datetime, timedelta
```

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/auth/token')

# Token creation
def create_access_token(data: dict, expires_delta: timedelta):
    payload = data.copy()
    payload['exp'] = datetime.utcnow() + expires_delta
    return jwt.encode(payload, SECRET_KEY, algorithm='HS256')

# Dependency: extract and validate JWT
async def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
        user_id = payload.get('sub')
        if not user_id:
            raise HTTPException(status_code=401, detail='Invalid token')
    except JWTError:
        raise HTTPException(status_code=401, detail='Invalid token')
    user = await db.get_user(user_id)
    if not user:
        raise HTTPException(status_code=401, detail='User not found')
    return user

# Role-based auth as a dependency factory
def require_role(role: str):
    def check_role(user = Depends(get_current_user)):
        if role not in user.roles:
            raise HTTPException(status_code=403, detail='Insufficient permissions')
        return user
    return check_role

@app.delete('/admin/user/{id}')
async def delete_user(id: int, user = Depends(require_role('admin'))):
    await db.delete_user(id)
```

Login endpoint: OAuth2PasswordRequestForm is a built-in Pydantic model for form-encoded username/password. The /auth/token endpoint validates credentials, returns access_token and refresh_token. Store the refresh token in an HttpOnly cookie; the access token in memory (not localStorage).

Q7: How do you handle exceptions and validation errors in FastAPI?

FastAPI has a layered exception handling system. Pydantic validation errors are caught and converted to 422 responses automatically. HTTPException is the standard for intentional API errors. Custom exception handlers let you add domain-specific error responses.

```
from fastapi import FastAPI, Request, HTTPException
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

app = FastAPI()

# Override default 422 validation error format
@app.exception_handler(RequestValidationError)
async def validation_error_handler(request: Request, exc: RequestValidationError):
    errors = []
    for error in exc.errors():
        errors.append({
            'field': '.'.join(str(x) for x in error['loc']),
            'message': error['msg'],
            'type': error['type'],
        })
```

```
    return JSONResponse(status_code=422, content={'errors': errors})

# Custom domain exception
class ResourceNotFound(Exception):
    def __init__(self, resource: str, id: int):
        self.resource = resource
        self.id = id

@app.exception_handler(ResourceNotFound)
async def not_found_handler(request: Request, exc: ResourceNotFound):
    return JSONResponse(status_code=404,
        content={'error': f'{exc.resource} {exc.id} not found'})

@app.get('/user/{id}')
async def get_user(id: int):
    user = await db.get(id)
    if not user:
        raise ResourceNotFound('User', id)  # clean domain exception
    return user
```

Global catch-all: Add an exception_handler for Exception to catch unhandled errors — log them with traceback, return a 500 response with a correlation ID, and never expose the internal error message to the client.

Q8: What are Background Tasks and when should you use them vs a message queue?

BackgroundTasks run after the HTTP response is sent, in the same process and event loop. They are lightweight and require no infrastructure. But they have real limitations.

```
from fastapi import BackgroundTasks

async def send_welcome_email(email: str, name: str):
    await email_client.send(
        to=email,
        subject='Welcome!',
        body=f'Hello {name}',
    )

@app.post('/register')
async def register_user(
    payload: UserCreate,
    background_tasks: BackgroundTasks,
):
    user = await db.create_user(payload)
    # Response sent to client immediately
    # Email sent after, client doesn't wait
    background_tasks.add_task(send_welcome_email, user.email, user.name)
    return {'id': user.id}
```

Limitations: If the process crashes before the task runs, the task is lost — no retry, no persistence. If the event loop is saturated, tasks are delayed. You cannot monitor or inspect scheduled tasks.

When to use a message queue instead: Use Celery + Redis/RabbitMQ or a Kafka producer when: (1) tasks must survive process restarts, (2) you need retries with backoff, (3) tasks are CPU-heavy and should run on separate workers, (4) you need task monitoring/progress reporting, (5) task volume is high enough to saturate the API process.

Rule of thumb: BackgroundTasks for: email sending, async logging, webhook delivery, cache invalidation — low-stakes, fast, fire-and-forget. Message queue for: video processing, PDF generation, payment callbacks, anything that must not be lost.

Q9: How do async and sync routes behave differently in FastAPI?

This is one of FastAPI's most misunderstood behaviors. Sync and async routes are NOT equivalent — they run in different execution contexts.

async def route: Runs directly on the event loop. Can use await. Blocking calls (sleep, requests.get, synchronous DB) will freeze the event loop for all other requests.

def route (sync): FastAPI automatically runs sync routes in a thread pool (via run_in_executor). This means they do NOT block the event loop. But they also cannot use await.

```
# This runs in thread pool — safe for blocking calls, no await allowed
@app.get('/sync')
def sync_route():
    time.sleep(1)          # OK — runs in thread, doesn't block loop
    return requests.get('http://...').json()   # OK — blocking but in thread

# This runs on event loop — must never block
@app.get('/async')
async def async_route():
    await asyncio.sleep(1)                # correct
    # time.sleep(1)                       # WRONG — freezes loop
    async with httpx.AsyncClient() as c:
        return await c.get('http://...').json()   # correct

# Common mistake: async route with synchronous ORM
@app.get('/wrong')
async def wrong():
    # SQLAlchemy sync session in async route — BLOCKS THE EVENT LOOP
    return db.query(User).all()
```

Decision rule: Use async def when you're doing async IO (async DB, httpx, redis-py async). Use sync def when you're calling blocking libraries you can't change. Never mix blocking calls into async def routes.

Q10: How do you scale a FastAPI application in production?

FastAPI itself is stateless and scales horizontally. The challenges are connection pooling, session management, shared state, and deploying multiple workers correctly.

Multiple workers: Run Uvicorn with Gunicorn as the process manager: gunicorn -w 4 -k uvicorn.workers.UvicornWorker main:app. Each worker is an independent process with its own event loop. 4 workers on a 4-core machine is typical. Each worker handles N concurrent requests via its event loop.

```
# gunicorn config (gunicorn.conf.py)
workers = 4                             # one per CPU core
worker_class = 'uvicorn.workers.UvicornWorker'
bind = '0.0.0.0:8000'
keepalive = 65                          # longer than load balancer timeout
timeout = 30                            # kill hung workers
graceful_timeout = 30                   # allow in-flight requests to complete
max_requests = 1000                     # restart workers to prevent memory leaks
max_requests_jitter = 100               # avoid all workers restarting at once
```

DB connection pooling: Each worker needs its own connection pool. With asyncpg, use create_pool() in startup event. Pool size per worker: typically 5-20 connections. Total connections = workers * pool_size. Monitor your DB's max_connections limit.

Shared state problem: Each worker process has its own memory. In-memory caches, rate limiters, and session stores are NOT shared between workers. Move shared state to Redis. Use redis-py (async) for rate limiting, session tokens, and distributed locks.

Kubernetes deployment: One pod = one Uvicorn worker (let K8s scale pods, not gunicorn workers). Set resource limits and readiness/liveness probes. Use HPA (Horizontal Pod Autoscaler) based on CPU or custom metrics (request latency).

```
# FastAPI startup/shutdown events for connection pool lifecycle
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: create DB pool, warm caches
    app.state.db_pool = await asyncpg.create_pool(DATABASE_URL, min_size=5,
max_size=20)
    app.state.redis = await aioredis.from_url(REDIS_URL)
    yield
    # Shutdown: close connections gracefully
    await app.state.db_pool.close()
    await app.state.redis.close()

app = FastAPI(lifespan=lifespan)
```

Q11: How do you structure a production FastAPI project?

Project structure should enforce separation of concerns and make testing easy. The router/service/repository pattern is idiomatic for FastAPI at production scale.

```
myapi/
├── app/
│   ├── main.py                # FastAPI app init, lifespan, middleware
│   ├── core/
│   │   ├── config.py          # Pydantic Settings (env vars)
│   │   ├── security.py        # JWT, password hashing
│   │   └── dependencies.py    # get_db, get_current_user
│   ├── api/
│   │   ├── v1/
│   │   │   ├── router.py      # includes all v1 sub-routers
│   │   │   ├── users.py       # user endpoints
│   │   │   └── auth.py        # auth endpoints
│   ├── models/
│   │   └── user.py            # SQLAlchemy models
│   ├── schemas/
│   │   └── user.py            # Pydantic schemas (in/out)
│   ├── services/
│   │   └── user_service.py    # business logic
│   ├── repositories/
│   │   └── user_repo.py       # DB queries
│   └── middleware/
│       └── logging.py
├── tests/
├── alembic/                   # migrations
└── docker-compose.yml
```

Config with Pydantic Settings: Use pydantic-settings BaseSettings to load from environment variables with type safety. All config in one place, validated at startup — not scattered os.getenv() calls across the codebase.

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    redis_url: str = 'redis://localhost:6379'
    debug: bool = False
    allowed_origins: list[str] = ['http://localhost:3000']
```

```
    model_config = ConfigDict(env_file='.env')

settings = Settings()   # raises on startup if required vars missing
```

Q12: How do you test FastAPI endpoints properly?

FastAPI provides TestClient (sync, wraps httpx) and AsyncClient for async tests. The key is dependency overriding — replace production dependencies (DB, auth, external APIs) with test doubles at the app level.

```
from fastapi.testclient import TestClient
import pytest, pytest_asyncio
from httpx import AsyncClient, ASGITransport

# Override production DB with test DB
def override_get_db():
    db = TestSessionLocal()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

# Sync test
def test_get_user():
    client = TestClient(app)
    response = client.get('/user/1')
    assert response.status_code == 200
    assert response.json()['id'] == 1

# Async test
@pytest.mark.asyncio
async def test_create_user():
    async with AsyncClient(transport=ASGITransport(app=app),
                           base_url='http://test') as client:
        response = await client.post('/user', json={'name': 'Gautham', 'email':
'g@test.com'})
        assert response.status_code == 201
```

Override current user: For auth-protected endpoints, override get_current_user to return a mock user. Never test against real JWTs — your test should control the auth identity completely.
